

1. What is the problem you want to solve? Why is it an interesting problem?

Problem: Nutritional labels are intentionally complex, heavily regulated, and often misleading, making it difficult for the average consumer to quickly determine if a food product is genuinely healthy. Existing scoring systems often fail to account for the nuance of ingredient quality (e.g., artificial additives, dyes) and metabolic impacts like "net carbs." Why it's interesting: Healthy eating directly impacts public health and preventative care. Automating complex nutritional analysis into a simple, highly accurate 1-10 score democratizes access to dietary guidance. It presents an interesting technical challenge because it requires combining strict tabular numerical data (macros) with unstructured text data (ingredient lists).

2. What data are you going to use to solve this problem? How will you acquire this data?

Data & Acquisition: The project utilizes the Open Food Facts database, a massive, crowdsourced, open data ecosystem of global food products.

- For Inference/Live App: Data is acquired dynamically in real-time using the Open Food Facts REST API via barcode scanning.
- For Model Training: The data will be acquired by downloading the Open Food Facts bulk data export (available in CSV/JSON formats), which contains hundreds of thousands of labeled food items, complete with macro-nutritional values and text-based ingredient lists.

3. Outline your approach to solving this problem.

a. Is this a supervised or unsupervised problem? This is a supervised learning problem, as we will train the model using labeled examples (historical foods previously scored or expert-labeled baseline foods). **b.** If supervised, is it a classification or regression problem? It is a regression problem, as the goal is to predict a continuous numerical value (a score ranging from 1.0 to 10.0). (Note: It could alternatively be framed as ordinal classification if bucketed into tiers like "Poor, Moderate, Excellent"). **c.** What are you trying to predict? We are predicting a holistic "Health Score" (1-10) that evaluates a food item's overall nutritional quality and safety. **d.** What will you use as predictors?

Predictors (features) will include:

- Numerical/Tabular: Total calories, protein, carbohydrates, dietary fiber, and total fat.
 - Derived/Engineered: Net carbs, protein-to-calorie ratio, and fiber density.
 - Text/Categorical: NLP-extracted features from the ingredients list (e.g., presence of hazardous additives like "red 40", or healthy indicators like "extra virgin olive oil" using TF-IDF or Word Embeddings).
- e.** Will you try a more "traditional" machine learning approach, a deep learning approach, or both? We will start with

a traditional machine learning approach (such as Random Forest, Gradient Boosting, or XGBoost). Tabular nutritional data and specific keyword flags perform exceptionally well with tree-based models, and they offer high interpretability (which is critical for the app's "Score Breakdown" feature to explain why a food got its score). We may later explore Deep Learning (NLP) specifically for parsing highly complex or misspelled ingredient strings.

4. What will be your final deliverable?

The final deliverable is a fully functional Progressive Web App (PWA) wrapped as a mobile application (Android APK / iOS installable). It features a native device-camera barcode scanner that fetches data via an API, processes it through the scoring algorithm, and displays a user-friendly UI with the final score and a line-by-line breakdown of the results.

5. What computational resources would you need at a minimum to do this project?

a. Processing power (CPU): A standard modern multi-core CPU (e.g., 4 to 8 cores) is required for data preprocessing, cleaning the massive Open Food Facts dataset, and training traditional tree-based models (XGBoost/Random Forest). b. Memory: 16GB to 32GB of RAM is recommended. The Open Food Facts bulk CSV dataset is quite large (several gigabytes uncompressed). High memory is necessary to load, clean, and manipulate this data in memory using libraries like Pandas without crashing. c. Specialized hardware such as GPUs: A GPU is not strictly required for the baseline traditional ML approach. However, access to a basic GPU (such as a free-tier Google Colab NVIDIA T4) would be highly beneficial if we utilize GPU-accelerated XGBoost to speed up training times, or if we pivot to using Deep Learning NLP models to analyze the ingredient texts. Once deployed, the actual inference on the mobile app is lightweight and requires zero specialized hardware.